

Software Design and Development Coursework (Midterm)

Part 1: Module coupling and cohesion

Qn A: identify two different types of module coupling that are happening in the Code

```
//start game
startButton.addEventListener("click", () => {
  startScreen.classList.add("hide");
  startGame();
});
```

One example of module coupling identified in the above code extract is **Control Coupling**. Module coupling by definition shows how software modules are interdependent on one another and how closely connected (i.e., strength of relationship) the modules are: High coupling – modules are connected closely, any change in one module will affect the other connected module. Low coupling – modules are independent of one another and any modification in one module will have little impact on another module.

In control coupling, a software module passes information to another module to direct the other module's execution. In the example code shown, the event listener of the "startButton" controls how the "startScreen" is shown and how the "startGame" function is triggered. There is dependency on the "startButton" event listener by the "startScreen" and "startGame" function.

This type of control coupling can be good when it comes to human interaction of the game. With the "startButton" event listener managing the flow of the software, there is more control and is necessary when there is user interaction. This type of coupling is considered high.

```

//Sets the circle to exact points
const setPiece = (startCount, colValue) => {
  let rows = document.querySelectorAll(".grid-row");
  //Initially it will place the circles in the last row else if no place available
  if (initialMatrix[startCount][colValue] !== 0) {
    startCount -= 1;
    setPiece(startCount, colValue);
  } else {
    //place circle
    let currentRow = rows[startCount].querySelectorAll(".grid-box");
    currentRow[colValue].classList.add("filled", `player${currentPlayer}`);
    //Update Matrix
    initialMatrix[startCount][colValue] = currentPlayer;
    //Check for wins
    if (winCheck(startCount, colValue)) {
      message.innerHTML = `Player<span> ${currentPlayer}</span> wins`;
      startScreen.classList.remove("hide");
      return false;
    }
  }
}
//Check if all are full
gameOverCheck();
};

```

The second type of module coupling to be discussed here is **Data Coupling**. This kind of coupling, aka. Input-Output coupling, takes place when one module feeds data (output) to another module (input), hence the name. The relationship between the modules is defined as only passing data to one another. This is considered to be a good type of coupling, if not the best type. As example of this coupling is extracted from the connect-4 game code.

In the “setPiece” function, there is a data sharing between it and “currentPlayer”. The “setPiece” function requires data from “currentPlayer” to decide which class (i.e., Player 1 or Player 2) is to be added to the HTML. “currentPlayer” essentially provides data to “setPiece” to let it know which player’s turn it is. There is a high level of coupling in this extract.

Qn B: identify two different types of module cohesion that are happening in the code

Module cohesion shows how strong elements or responsibilities in a module are related, the degree of how the elements work together within a module to perform a single task. The higher the cohesion the better.

```
//When user clicks on a box
const fillBox = (e) => {
  //get column value
  let colValue = parseInt(e.target.getAttribute("data-value"));
  //5 because we have 6 rows (0-5)
  setPiece(5, colValue);
  currentPlayer = currentPlayer == 1 ? 2 : 1;

  playerTurn.innerHTML = `Player <span>${currentPlayer}'s</span> turn`;
};
```

One type of module cohesion spotted in the common-4 game code is **Sequential cohesion**. This type of cohesion happens when elements of a module are in a particular sequence, where one element produces an output which becomes the input for the next element in a step-by-step format.

In the above extract from the codes, the “fillBox” function has a sequence of elements which starts with a user clicking on a box in the game. The column value is then derived and then triggers the next step of “setPiece” function which takes in the column value together with the row index to set the piece in the game grid. “currentPlayer” is then activated to determine player turn.

The elements above show a strict sequence of execution of the elements within a module to make sure that the game state is updated accurately after a player has its turn. There is a high level of cohesion here.

```

//Check diagonal
const checkAdjacentDiagonalValues = (row, column) => {
  let diagWinBool = false;
  let tempChecks = {
    leftTop: [],
    rightTop: [],
  };
  let columnLength = initialMatrix[row].length;
  let rowLength = initialMatrix.length;

  //Store left and right diagonal array
  tempChecks.leftTop = [
    ...getLeftDiagonal(row, column, rowLength, columnLength),
  ];

  tempChecks.rightTop = [
    ...getRightDiagonal(row, column, rowLength, columnLength),
  ];
  //check both arrays for similarities
  diagWinBool = verifyArray(tempChecks.rightTop);
  if (!diagWinBool) {
    diagWinBool = verifyArray(tempChecks.leftTop);
  }
  return diagWinBool;
};

```

Here in the extracted code, we see an instance of **Communicational Cohesion** – tasks performed by a module operate on the same input data or contribute towards producing the same output data. The shared input data in the above code are “row” and “column” parameters and are used by both “getLeftDiagonal” and “getRightDiagonal” to produce towards the same output data stored in “tempChecks” which will ultimately determine if there’s a win in a diagonal manner in the game. Communicational cohesion shows high cohesion and produces focused modules with clear purposes – i.e., finding diagonal wins in the example extract above.